



# Документно-орієнтовані бази даних



```
{ "id" : "13434",  
  "value1" : "sfds"  
  "value2" : "sfds"  
  "items" : [ { "_id" : "3fef2",  
    "t2value" : "abcd", .. } ] }
```

Документно-орієнтовані бази даних є одним із типів NoSQL баз даних, які зберігають дані у вигляді документів.

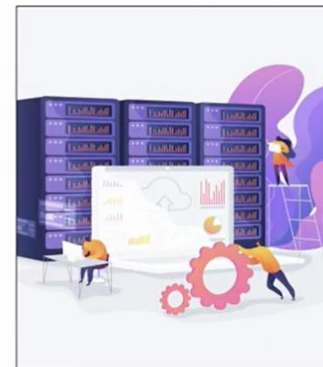
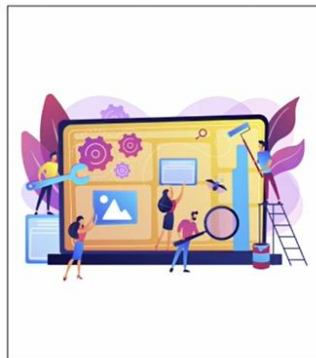
Ці документи зазвичай мають формат JSON (JavaScript Object Notation) або BSON (Binary JSON), що робить їх гнучкими та легко читабельними.

Документно-орієнтовані бази даних забезпечують багаті можливості запитів, дозволяючи виконувати відносно складні запити та індексувати поля всередині документів.

Вони також можуть масштабуватися горизонтально, розподіляючи дані між кількома вузлами.

# Використання документно-орієнтованих баз даних

Документно-орієнтовані бази даних вважаються універсальними, подібно до швейцарського ножа серед NoSQL баз даних, і вони корисні для широкого спектру потреб.



1

## CMSs

Content can easily be stored, retrieved, and filtered

2

## Large Document Storage

Files, general data, and more

3

## Websites

Great for storing multiple types of unstructured data

4

## User Profiles

Many different data types are able to be accessed and stored

5

## Real-Time Analytics

Allows developers to gain real-time access to operational data

6

## Big Data

Analyzes a lot of unstructured data

### Системи управління контентом (CMS)

Зберігання різних типів контенту (тексти, зображення, відео) у гнучкому форматі.

### Велике документне сховище

Ідеальні для зберігання великої кількості документів, таких як електронні листи, звіти, файли.

### Вебсайти

Зберігання даних вебсайтів, таких як профілі користувачів, публікації в блогах, коментарі.

### Профілі користувачів

Зберігання складних профілів користувачів з різними атрибутами (ім'я, електронна пошта, адреса, історія покупок).

### Аналітика в реальному часі

Зберігання та аналіз даних у реальному часі для прийняття рішень.

### Великі дані

Підходять для зберігання та обробки великих обсягів даних у проектах Big Data.

# Чому документно-орієнтовані бази даних універсальні?

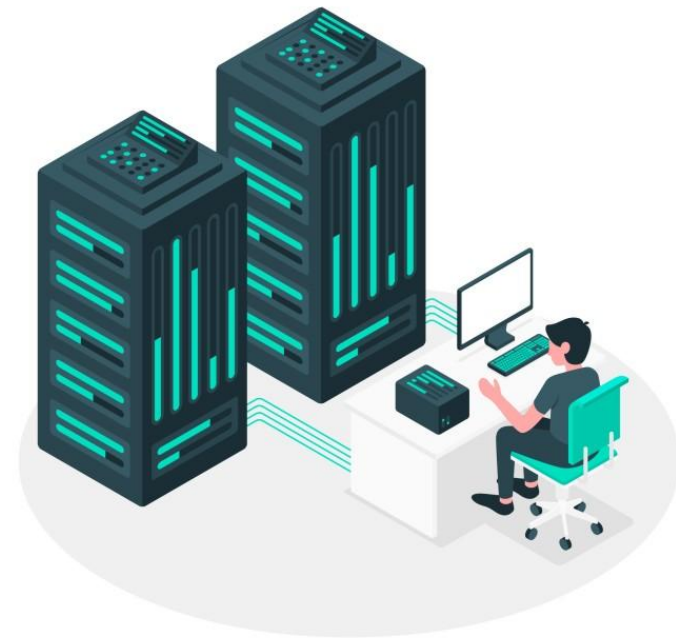
**Гнучка схема:** Документно-орієнтовані бази даних не вимагають попередньо визначеної схеми, що дозволяє легко змінювати структуру даних у міру розвитку проєкту.

**Зручність роботи з JSON:** Оскільки багато сучасних веб-додатків використовують JSON для обміну даними, документно-орієнтовані бази даних ідеально підходять для таких додатків.

**Індексація:** Можливість створення індексів на будь-якому полі документа для прискорення запитів.

**Масштабованість:** Вони підтримують горизонтальне масштабування, що дозволяє легко розподіляти дані і навантаження між кількома серверами.

**Висока продуктивність:** Оптимізовані для швидкого читання та запису даних, що робить їх ефективними для додатків з високими вимогами до продуктивності.



# Рейтинг документо-орієнтованих баз даних

| Rank     |          |          | DBMS                      | Database Model         | Score    |          |          |
|----------|----------|----------|---------------------------|------------------------|----------|----------|----------|
| Oct 2025 | Sep 2025 | Oct 2024 |                           |                        | Oct 2025 | Sep 2025 | Oct 2024 |
| 1.       | 1.       | 1.       | Redis                     | Key-value, Multi-model | 142.33   | -2.84    | -7.30    |
| 2.       | 2.       | 2.       | Amazon DynamoDB           | Multi-model            | 75.91    | -4.37    | +4.06    |
| 3.       | 3.       | 3.       | Microsoft Azure Cosmos DB | Multi-model            | 22.71    | -1.23    | -1.78    |
| 4.       | 4.       | 4.       | Memcached                 | Key-value              | 15.48    | -0.69    | -2.32    |
| 5.       | 5.       | 5.       | etcd                      | Key-value              | 7.35     | -0.10    | +0.17    |
| 6.       | 6.       | ↑ 7.     | Hazelcast                 | Key-value, Multi-model | 5.29     | +0.03    | -0.27    |
| 7.       | 7.       | ↓ 6.     | Aerospike                 | Multi-model            | 4.53     | -0.58    | -1.05    |
| 8.       | 8.       | ↑ 11.    | Oracle NoSQL              | Multi-model            | 3.35     | -0.27    | +0.08    |
| 9.       | ↑ 11.    | 9.       | Riak KV                   | Key-value              | 3.32     | 0.00     | -0.51    |
| 10.      | ↑ 12.    | ↑ 14.    | Apache Ignite             | Multi-model            | 2.99     | -0.11    | +0.05    |
| 11.      | ↓ 9.     | ↓ 8.     | Ehcache                   | Key-value              | 2.99     | -0.49    | -1.77    |
| 12.      | ↑ 13.    | ↓ 10.    | ArangoDB                  | Multi-model            | 2.91     | -0.02    | -0.52    |
| 13.      | ↓ 10.    | ↓ 12.    | RocksDB                   | Key-value              | 2.84     | -0.49    | -0.20    |
| 14.      | 14.      | ↑ 17.    | GemFire                   | Key-value, Multi-model | 2.79     | -0.14    | +0.18    |
| 15.      | 15.      | 15.      | Google Cloud Bigtable     | Multi-model            | 2.65     | -0.10    | -0.26    |
| 16.      | 16.      | ↓ 13.    | OrientDB                  | Multi-model            | 2.62     | +0.03    | -0.41    |
| 17.      | 17.      | ↓ 16.    | InterSystems IRIS         | Multi-model            | 2.20     | -0.07    | -0.69    |
| 18.      | 18.      | 18.      | LevelDB                   | Key-value              | 2.00     | -0.10    | -0.39    |
| 19.      | ↑ 20.    | ↑ 20.    | LMDB                      | Key-value              | 1.95     | -0.06    | +0.03    |
| 20.      | ↓ 19.    | ↓ 19.    | Infinispan                | Key-value              | 1.94     | -0.10    | -0.38    |
| 21.      | ↑ 22.    |          | Valkey                    | Key-value              | 1.80     | +0.06    |          |
| 22.      | ↓ 21.    | 22.      | Oracle Coherence          | Key-value              | 1.75     | -0.04    | -0.09    |
| 23.      | 23.      | 23.      | Amazon SimpleDB           | Key-value              | 1.70     | -0.01    | -0.12    |
| 24.      | 24.      | ↓ 21.    | Oracle Berkeley DB        | Multi-model            | 1.64     | -0.01    | -0.22    |
| 25.      | 25.      | ↑ 26.    | Geode                     | Key-value              | 1.38     | -0.02    | -0.06    |
| 26.      | 26.      | ↓ 25.    | GridGain                  | Multi-model            | 1.35     | -0.04    | -0.14    |
| 27.      | 27.      | ↓ 24.    | Tarantool                 | Multi-model            | 1.31     | -0.06    | -0.27    |
| 28.      | 28.      | ↓ 27.    | FoundationDB              | Multi-model            | 0.94     | -0.03    | -0.05    |
| 29.      | 29.      | ↓ 28.    | GT.M                      | Key-value              | 0.92     | +0.02    | -0.05    |
| 30.      | ↑ 31.    | ↓ 29.    | NCache                    | Key-value, Multi-model | 0.83     | +0.02    | -0.10    |
| 31.      | ↓ 30.    | ↓ 30.    | ZONDR                     | Key-value              | 0.80     | -0.03    | -0.01    |

## Популярні документо-орієнтовані бази даних

**MongoDB:** Найпопулярніша документо-орієнтована база, зберігає дані у BSON, забезпечує масштабованість і високу швидкість.

**Databricks:** Мульти-модельна платформа для аналітики та роботи з великими даними, підтримує документи.

**Amazon DynamoDB:** Керована NoSQL база від AWS з підтримкою документів і key-value моделі.

**Azure Cosmos DB:** Глобально розподілена мульти-модельна база від Microsoft із підтримкою документів.

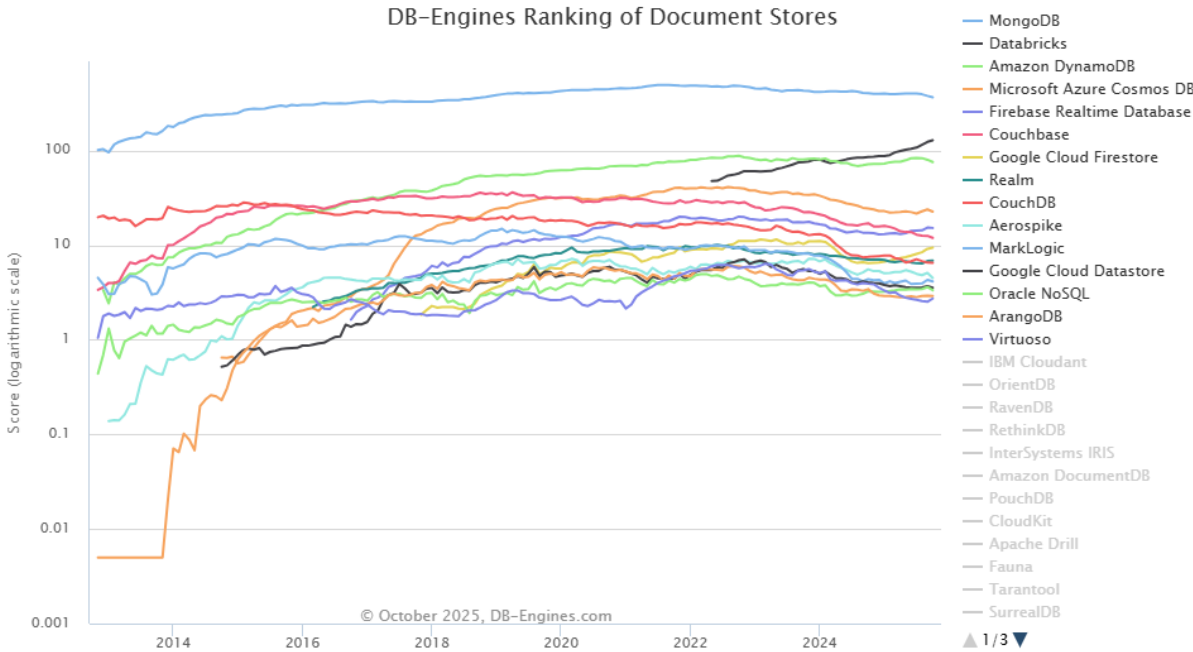
**Firebase Realtime Database:** Хмарна база від Google для синхронізації даних у реальному часі.

**Couchbase:** Мульти-модельна система, що поєднує документи, кешування та пошук.

**Google Cloud Firestore:** Документо-орієнтована хмарна база з автоматичною синхронізацією та масштабуванням.

**Realm:** Легка документо-орієнтована база для мобільних застосунків з офлайн-режимом.

**CouchDB:** Відкрита документо-орієнтована база з простою реплікацією через HTTP.



# Документно-орієнтовані бази даних



# MongoDB



**MongoDB** — це популярна система керування базами даних з відкритим кодом, яка належить до NoSQL та є документно-орієнтованою. Використовується для зберігання великих обсягів інформації. Код MongoDB написаний мовою програмування C++.

# MongoDB в порівнянні з реляційною базою даних

| MongoDB                                 | RDBMS                               |
|-----------------------------------------|-------------------------------------|
| Нереляційна база даних                  | Реляційна база даних                |
| Документно-орієнтована                  | Рядково-орієнтована                 |
| Поле-орієнтована                        | Стовпцево-орієнтована               |
| Колекційно-орієнтована                  | Таблично-орієнтована                |
| Клієнт на основі JavaScript для запитів | Вбудованого клієнта немає           |
| Майже не піддається SQL-ін'єкціям       | Легко піддається SQL-ін'єкціям      |
| Має динамічну схему                     | Попередньо визначена статична схема |
| Легко зберігати ієрархічні дані         | Важко зберігати ієрархічні дані     |
| Горизонтально масштабована              | Можливе вертикальне масштабування   |

# Архітектура MongoDB

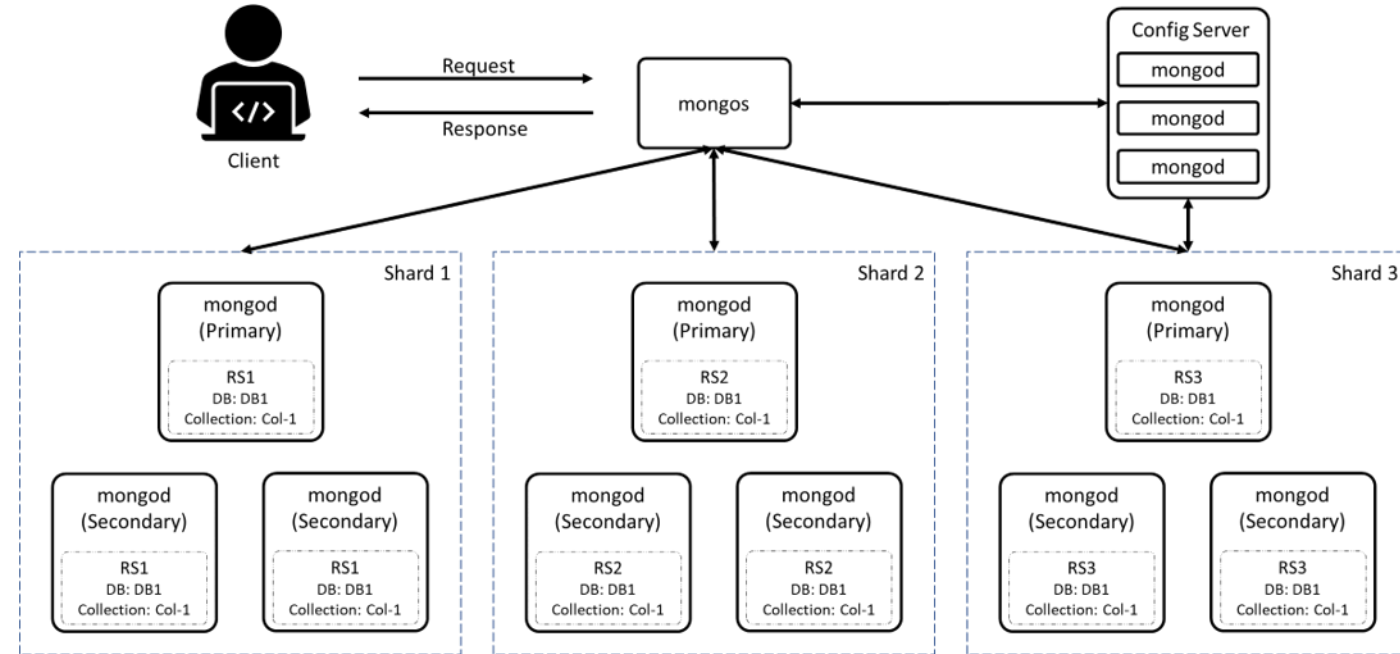
**Mongos** – це інтерфейс між **клієнтом** і **шардинг-кластерами**. Він підключається до **конфігураційних серверів**, щоб визначити, який **шард** повинен відповідати на запит.

**Конфігураційний сервер** містить метадані про частини даних у кожному шарді і підтримує з'єднання з іншими компонентами бази даних для забезпечення доступності даних.

**Шард (Шардинг)** – це розподіл даних між кількома машинами для масштабування MongoDB.

Існує два методи масштабування: вертикальне та горизонтальне.

Кожен шард розгортається як **набір реплік**, що забезпечує автоматичне перемикання у разі відмови і забезпечує реплікацію даних для підвищення надійності та доступності.



Архітектура MongoDB

# Приклад документа MongoDB

```
{
  _id: ObjectId(6c38d47de4c3)
  title: 'Introduction to MongoDB',
  description: 'This is the MongoDB Example document',
  by: 'DigitalVarys',
  url: 'http://digitalvarys.com',
  tags: ['mongodb', 'database', 'NoSQL'],
  Topics: [
    {
      Title: 'Introduction',
      Description: 'Introduction of the MongoDB',
    },
    {
      Title: 'Support Platform and Community',
      Description: 'Links to all MongoDB Community',
    },
    {
      Title: 'Concepts of MongoDB.',
      Description: 'Explains the Basic concept of MongoDB',
    },
    {
      Title: 'Sharding of MongoDB',
      Description: 'Explains the concept of Replication',
    },
    {
      Title: 'Importance of MongoDB Technology',
      Description: 'Explains the importance and advantages of MongoDB',
    }
  ]
}
```

`_id` – це унікальний 12-байтовий шістнадцятковий номер. Він подібний до ID у реляційних базах даних (RDBMS).

Інші поля є парами "ключ-значення" даних.

*Ви можете вказати `_id` під час вставлення документа.*

*Якщо не вказати, тоді MongoDB генерує унікальний ідентифікатор для кожного документа.*

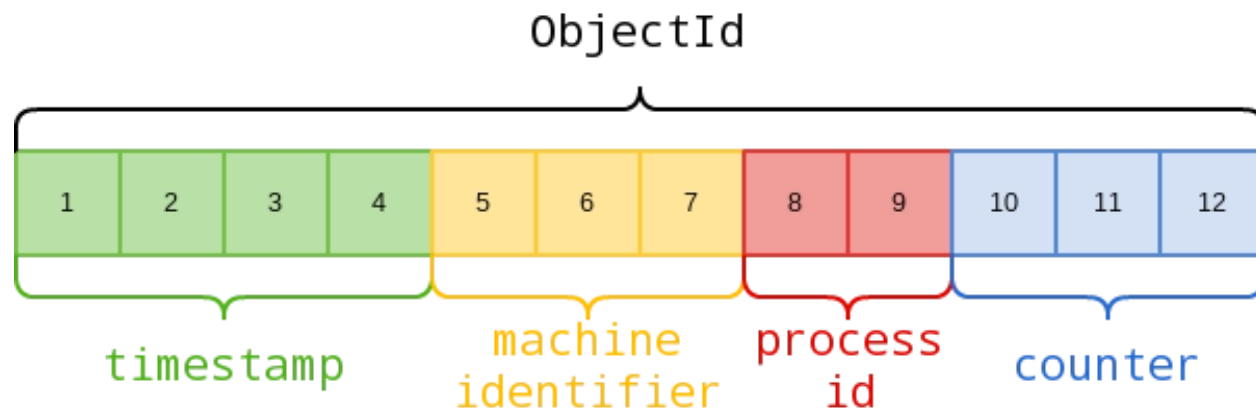
## 12 байт:

**перші 4 байти** для поточної позначки часу,

**наступні 3 байти** для машинного ідентифікатора,

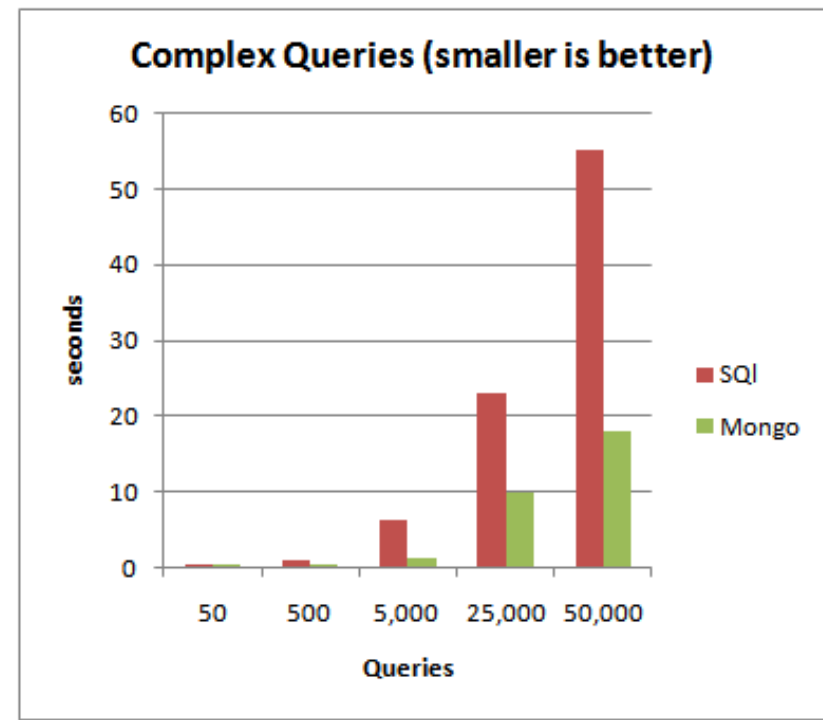
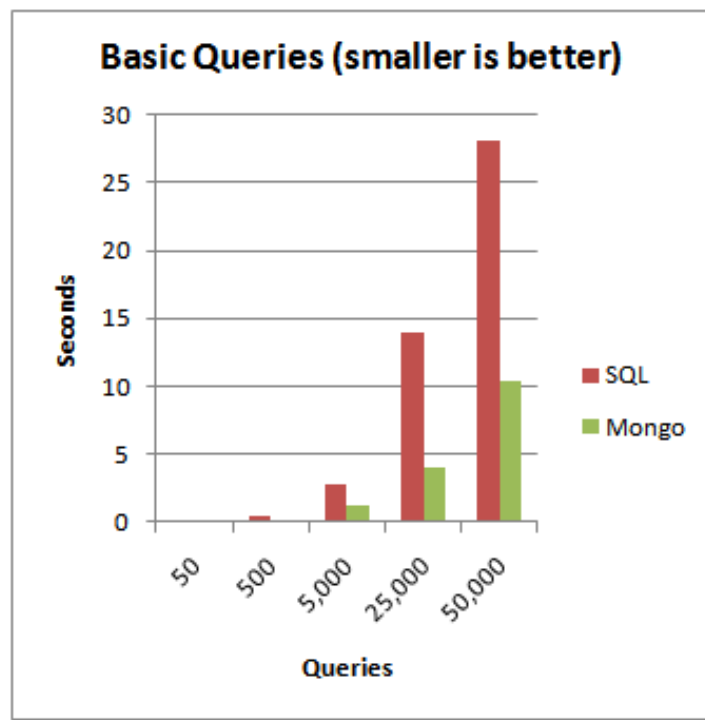
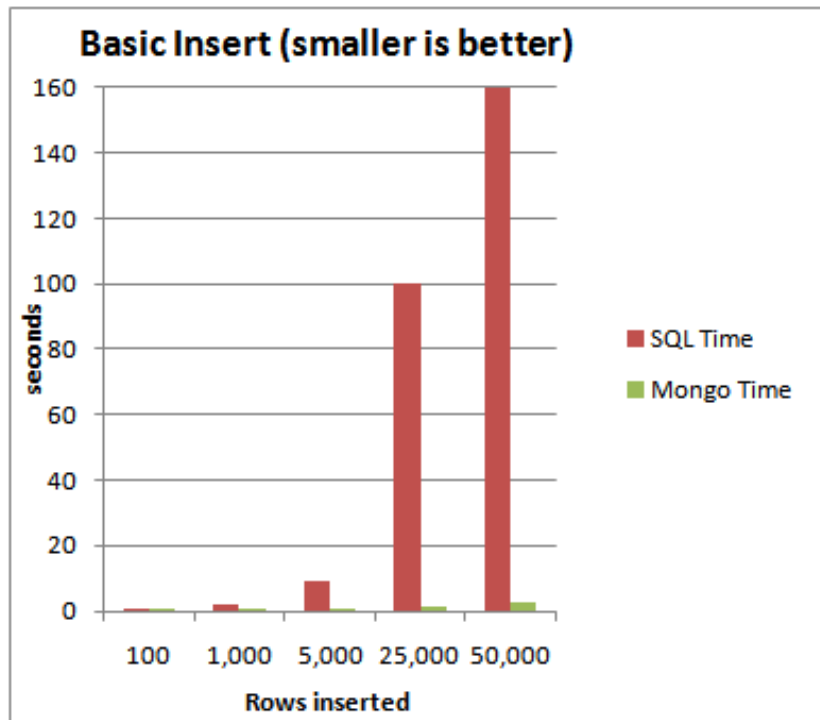
**наступні 2 байти** для ідентифікатора процесу сервера MongoDB

**решта 3 байти** - просте додаткове значення.



# Продуктивність – MongoDB vs SQL Server

- вставлення 50 000 незалежних об'єктів за допомогою NoRM для MongoDB та LINQ в SQL для SQL Server 2008
- п'ять одночасних клієнтів
- запити в індексованих полях id



- Hardware : Lenovo T61
- Windows 7 64 bit / Dual Core 2.8 Ghz
- SQL Server 2008 Standard & MongoDB 1.4.1

*NoRM (NoSQL Object Relational Mapper) – це бібліотека для .NET, яка дозволяє взаємодіяти з MongoDB.  
LINQ to SQL – це компонент .NET, який дозволяє взаємодіяти з SQL Server за допомогою LINQ-запитів.*

дані за 2013 рік

[lovedb.tistory.com/159](http://lovedb.tistory.com/159)

# Важливість технології MongoDB

**MongoDB має багато переваг над традиційними базами даних, серед яких:**

## **Масштабованість:**

Легко розподіляє дані по всьому світу.

## **Пошук по тексту:**

Вбудована функція для зручного пошуку.

## **Гнучка модель даних:**

Підтримує динамічну схему, що дозволяє змінювати структуру даних у будь-який момент.

## **Швидка та ітеративна розробка:**

Підтримка змін та адаптації прискорює розробку.

## **Інтегровані функції:**

Аналітика, візуалізація, потокова обробка, пошук, обробка графів, продуктивність у пам'яті та безпека без додаткових зусиль.

## **Економічність:**

Рентабельна та економічна в обслуговуванні.

# Важливість технології MongoDB

## Навіщо використовувати MongoDB?

- ✓ Документо-орієнтоване зберігання - дані зберігаються у вигляді документів у стилі JSON.
- ✓ Індексуйте будь-який атрибут
- ✓ Тиражування та висока доступність
- ✓ Швидке оновлення на місці
- ✓ Професійна підтримка MongoDB

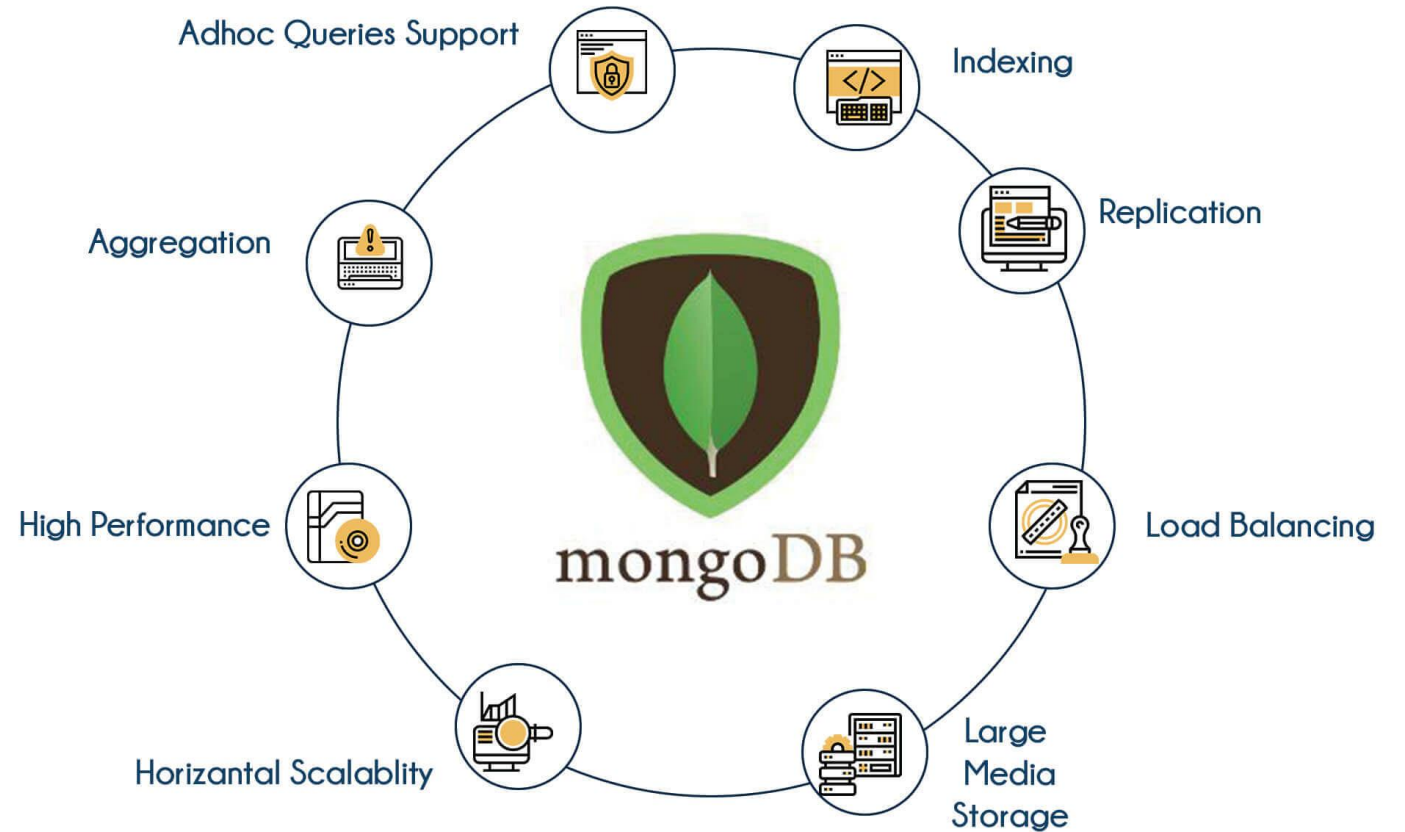
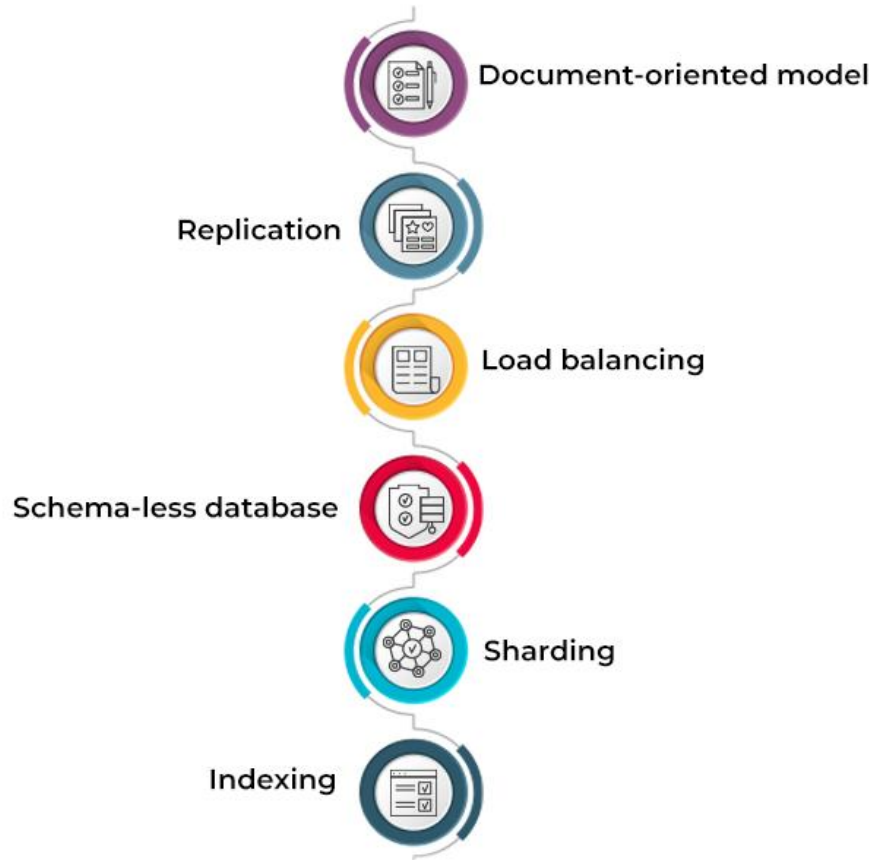
## Де використовувати MongoDB?

- ✓ Великі дані
- ✓ Управління контентом і доставка
- ✓ Мобільна та соціальна інфраструктура
- ✓ Управління даними користувачів
- ✓ Центр даних



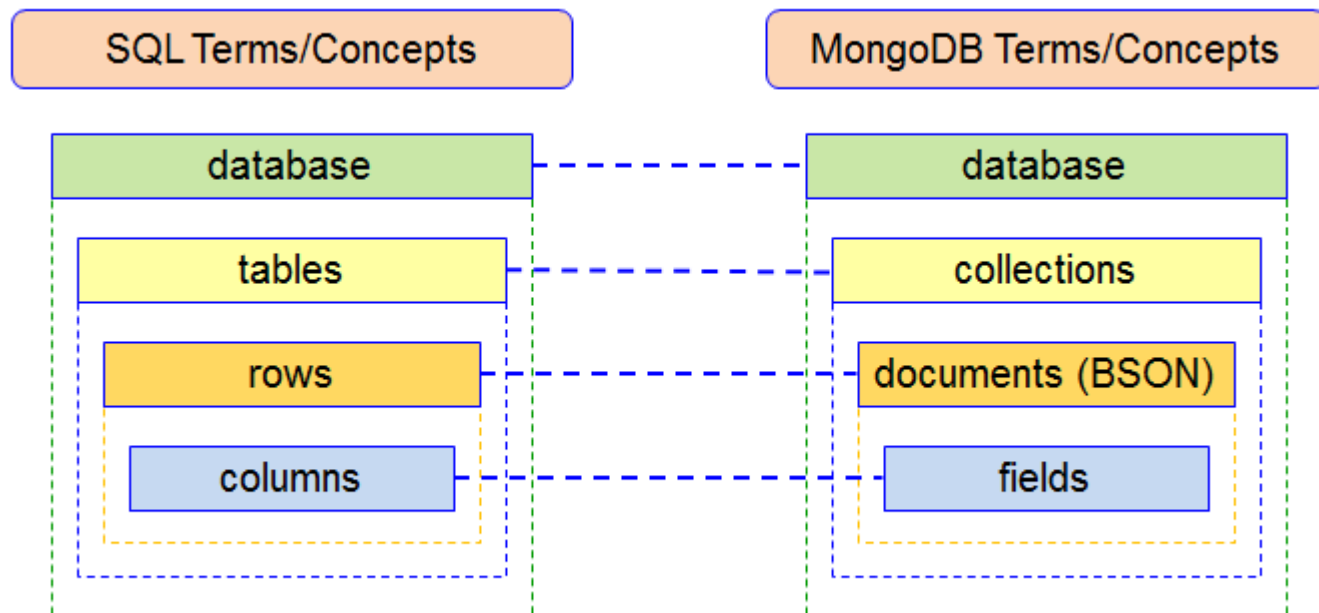
# Основні можливості MongoDB

## KEY FEATURES OF MONGODB



# Невеличке порівняння SQL з MongoDB

Багато компонентів, які входять до структури даних MongoDB, мають аналоги в системах керування реляційними базами даних на основі мови структурованих запитів (SQL).



MongoDB складається з баз даних, які зберігають в собі колекції.

Кожна колекція складається з документів, які в свою чергу складаються з полів.

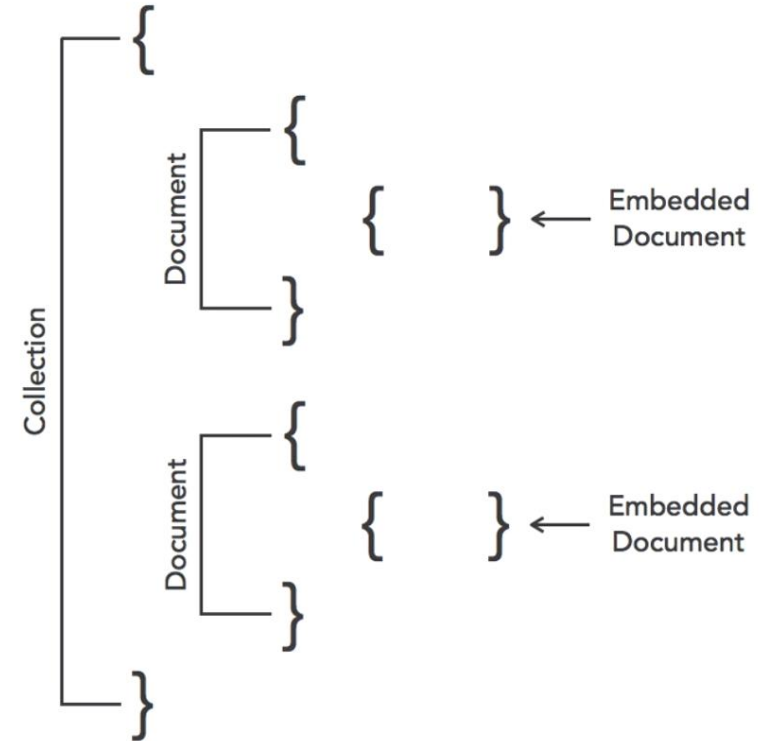
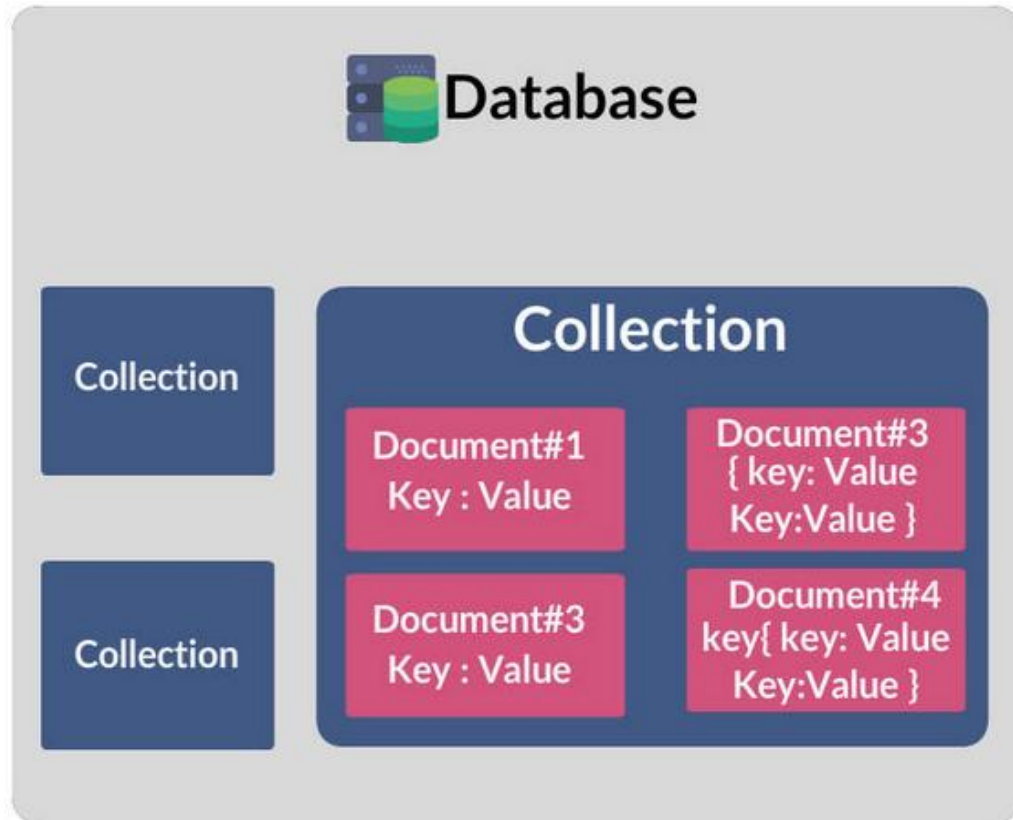
Поля є парами ключ-значення.

Колекції також можуть бути індексовані, що покращує швидкість вибірки та сортування.

# Концепції MongoDB

MongoDB працює на основі таких концепцій:

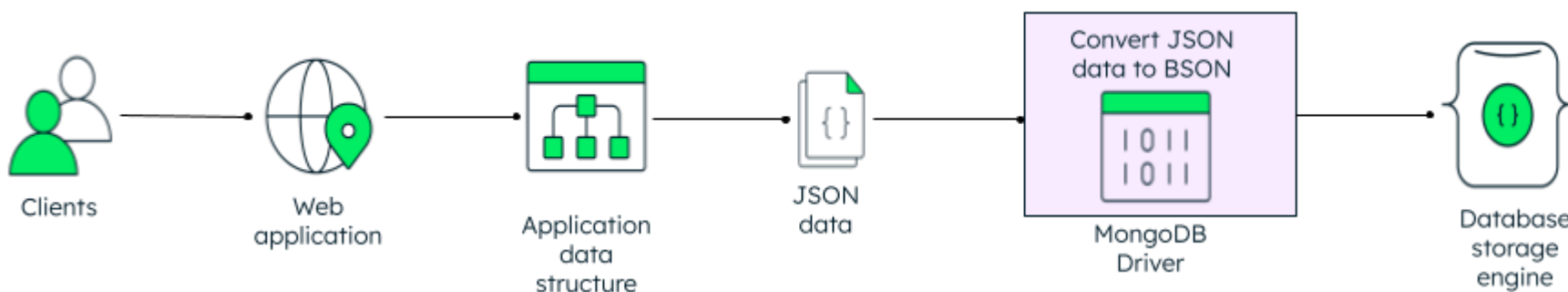
- **База даних** — фізичний контейнер для даних.
- **Колекції** — це контейнери для структурно чи концептуально подібних документів.
- **Документ** — це набір пар "ключ-значення" (фактично у форматі JSON).



# Binary JavaScript Object Notation (BSON)

**BSON (Binary JSON)** – це формат серіалізації даних, який використовується MongoDB для зберігання документів.

Є бінарним представленням JSON-подібних документів, що забезпечує ефективне зберігання та швидкий доступ до даних. Пошук по BSON здійснюється набагато швидше.



**JSON** – це широко використовуваний формат обміну даними, популярний у багатьох додатках і технологічних стеків.

**BSON**, бінарне представлення JSON, використовується в основному внутрішньо в MongoDB для ефективного зберігання та обробки даних.

## JSON

```
{
  "array": [
    1,
    2,
    3
  ],
  "boolean": true,
  "null": null,
  "number": 123,
  "object": {
    "a": "b",
    "c": "d",
    "e": "f"
  },
  "string": "Hello World"
}
```

## BSON

```
{
  01010100
  11101011
  10101110
  01010101
}
```

```
{"hello": "world"} →
\x16\x00\x00\x00 // total document size
\x02              // 0x02 = type String
hello\x00         // field name
\x06\x00\x00\x00world\x00 // field value
\x00              // 0x00 = type E00 ('end of object')
```

# Перше знайомство з MongoDB

# Інсталяція MongoDB

**MongoDB Community Edition** – база даних з відкритим кодом.

[mongodb.com/try/download/community](https://mongodb.com/try/download/community)

**MongoDB Shell** (наприклад для windows) – клієнт для взаємодії із сервером MongoDB через командний рядок

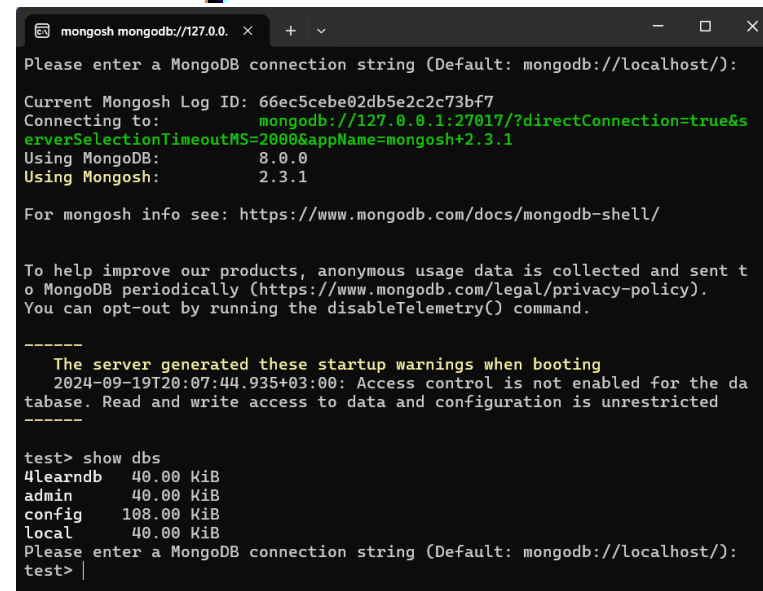
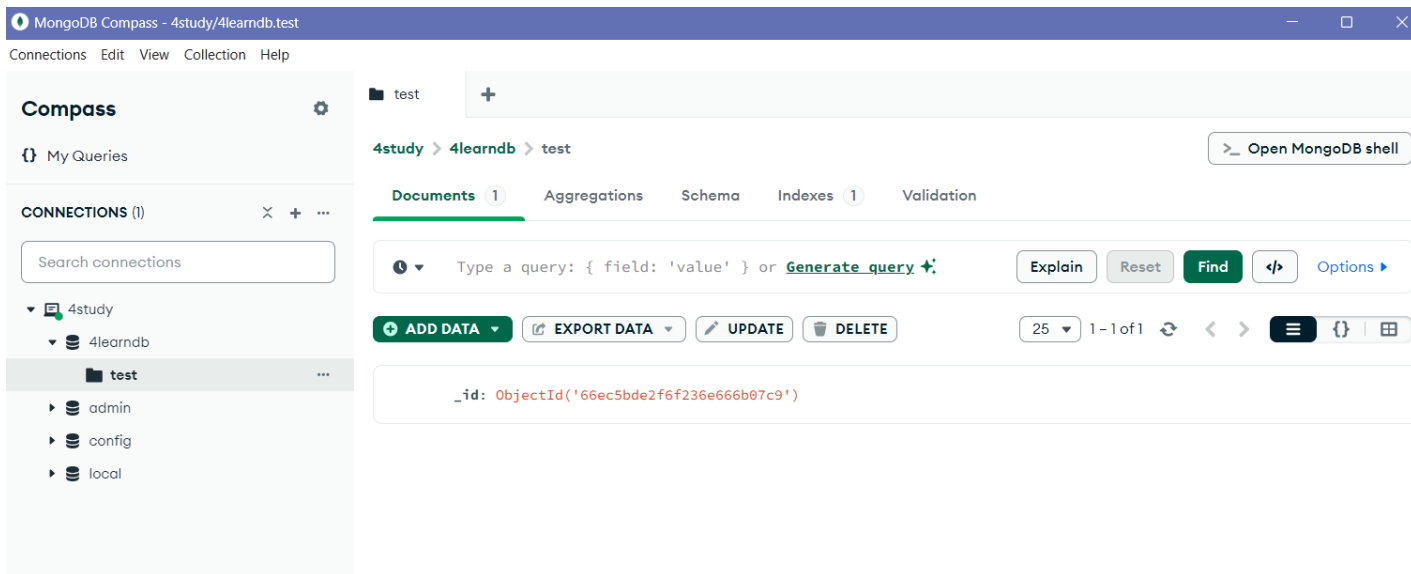
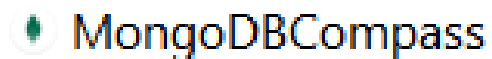
[mongodb.com/try/download/shell](https://mongodb.com/try/download/shell)

**Compass** – графічний інтерфейс для роботи з MongoDB.

[mongodb.com/products/tools/compass](https://mongodb.com/products/tools/compass)

**MongoDB Atlas** – хмарна база даних

[mongodb.com/atlas](https://mongodb.com/atlas)



# MongoDB Shell

**Mongo-шелл (mongosh)** — це інтерактивна JavaScript оболонка до MongoDB.

Ви можете використовувати її для створення та зміни даних, а також і для адміністрування.

## Створення та видалення БД

Для зміни поточної БД, а також для створення нової, використовують команду **use Database\_Name**.

Команда **db** виводить поточну БД, а **show dbs** – всі існуючі бази даних.

Видалити БД можна командою **db.dropDatabase()**

## Створення та видалення колекцій

Створення колекцій відбувається за допомогою команди: **db.createCollection(name, options)**

Тут **name** - ім'я для колекції, а **options** (не обов'язковий параметр) зберігає опції індексації та ліміту пам'яті.

**db.createCollection("myCollection", { capped: true, size: 5242880, max: 5000 })**

Видалення колекцій: **db.COLLECTION\_NAME.drop()**

**db.myCollection.drop()**

Колекції також можна перейменувати: **db.COLLECTION\_NAME.renameCollection("newName")**

# MongoDB Shell – CRUD операції

## Створення документів

Методи **insertOne()** та **insertMany()** дозволяють додавати нові документи в колекцію.

Також, якщо викликати вставку на неіснуючій колекції, то вона буде створена і в неї буде додано цей документ.

```
db.employees.insertOne({  
  "employee_id":1,  
  "name":"employee1"  
});
```

```
db.employees.insertMany([  
  { "employee_id": 2, "name": "employee2" },  
  { "employee_id": 3, "name": "employee3" }  
]);
```

Тут **employees** це колекція, що зберігає об'єкти **employee**.

# MongoDB Shell – CRUD операції

## Пошук документів (читання)

Щоб знайти потрібний документ, слід виконати метод **find()**.

Також можна опціонально додати виклик методу **pretty()**, що зробить вивід більш читабельним.

```
db.employees.find().pretty()
```

```
{
  "_id" : ObjectId("579c657b56c7d42812e2788c"),
  "employeeid" : 1,
  "name" : "Andrew"
}
{
  "_id" : ObjectId("579c67e9b87b4b49be12664b"),
  "name" : "employee1",
  "employee_id" : 2
}
```

Зверніть увагу на поле **\_id** — це 12-байтовий ідентифікатор у шістнадцятковій системі, який є унікальним для кожного документа.

# MongoDB Shell – CRUD операції

## Оновлення записів

У MongoDB є кілька методів для оновлення документів: **updateOne()**, **updateMany()**, та **replaceOne()**.

Метод **updateOne()** оновлює один документ, який відповідає критеріям пошуку.

```
db.employees.updateOne(  
  { "employee_id": 2 },  
  { $set: { "name": "Employee2" } }  
);
```

Метод **updateMany()** оновлює всі документи, які відповідають критеріям пошуку.

```
db.employees.updateMany(  
  { "department": "Sales" },  
  { $set: { "department": "Marketing" } }  
);
```

Метод **replaceOne()** замінює один документ, який відповідає критеріям пошуку, на новий документ.

```
db.employees.replaceOne(  
  { "employee_id": 1 },  
  { "employee_id": 1, "name": "Michele" }  
);
```

# MongoDB Shell – CRUD операції

## Видалення документа

Для видалення документів використовуються методи **deleteOne()** та **deleteMany()**.

Метод **deleteOne()** видаляє один документ, який відповідає критеріям пошуку.

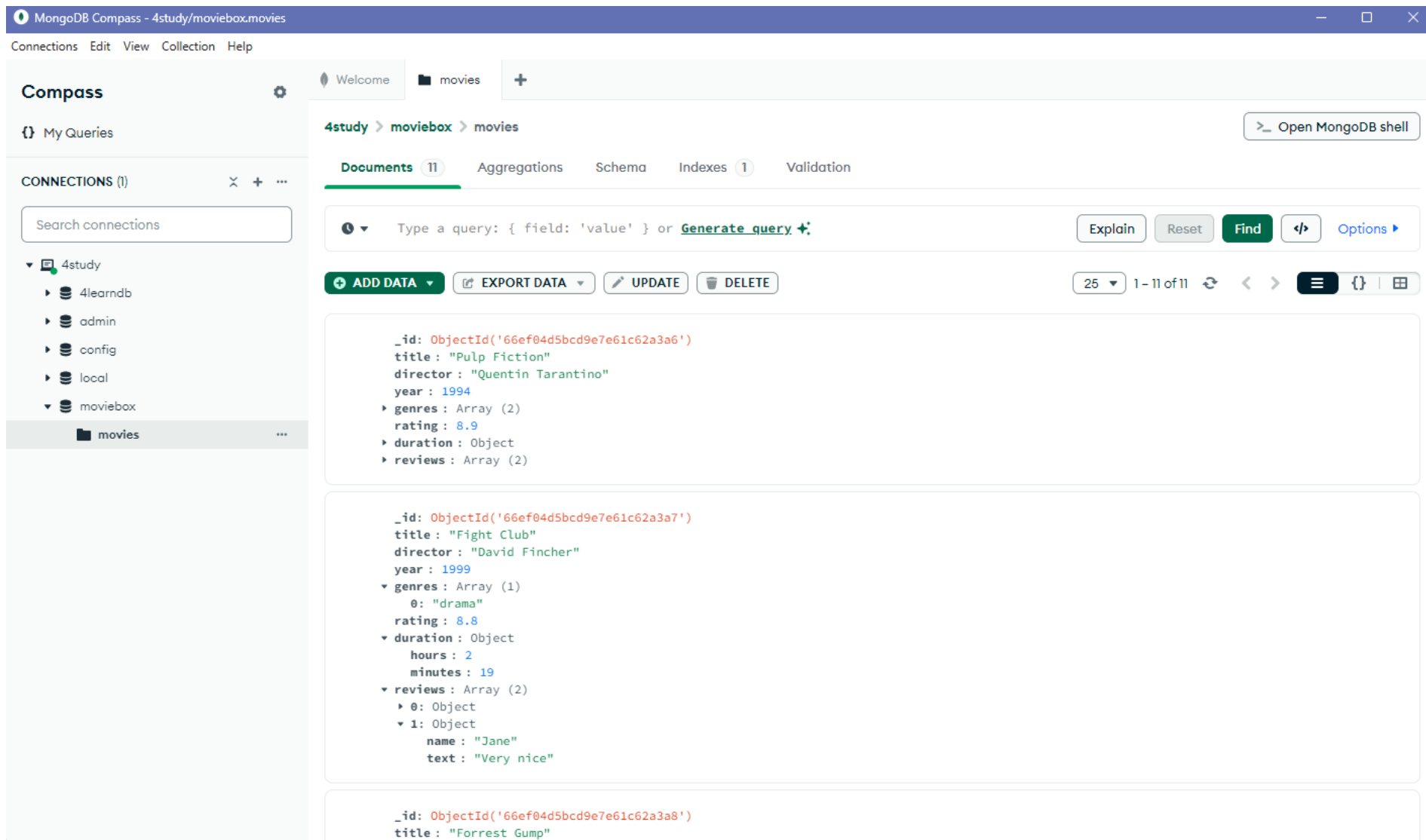
```
db.employees.deleteOne({ "_id": ObjectId("579c6ecab87b4b49be12664c") });
```

Метод **deleteMany()** видаляє всі документи, які відповідають критеріям пошуку.

```
db.employees.deleteMany({ "department": "Sales" });
```

# MongoDB Compass

**MongoDB Compass** – це графічний інтерфейс для роботи з базою даних MongoDB. Він надає зручний спосіб взаємодії з вашими даними, дозволяючи виконувати різні операції без необхідності писати код.



# Використання MongoDB

MongoDB має широке застосування як система баз даних.

**Деякі реальні приклади використання включають:**

**MetLife:** Глобальна компанія, яка надає програми соціального забезпечення, включаючи страхування, ануїтети тощо. Використовує MongoDB для додатку обслуговування клієнтів "The Wall", що обслуговує 90 млн осіб у 60 країнах.

**Forbes:** Розробив мобільний додаток і переробив вебсайти за допомогою MongoDB для надання динамічного контенту.

**Otto:** Одна з найбільших у світі компаній електронної комерції з великою клієнтською базою в понад 20 країнах. Зменшив час відгуку до 1-2 секунд, використовуючи MongoDB для аналітики в реальному часі.

**Shutterfly:** Компанія з обміну цифровими зображеннями та фотографіями. Перейшов на MongoDB для управління 6 млрд зображень, обробляючи понад 10 000 зображень на секунду.

**UIDAI:** Використовує MongoDB для зберігання біометричної інформації понад 1 млрд індійців.

**eBay:** Міжнародна компанія електронної комерції. Використовує MongoDB для управління хмарними обчисленнями, пошуковими підказками та категоризацією товарів.

# Команди в MongoDB з прикладом

# Робота з MongoDB

```
[
  {
    "title": "Pulp Fiction",
    "director": "Quentin Tarantino",
    "year": 1994,
    "genres": ["crime", "drama"],
    "rating": 8.9,
    "duration": {
      "hours": 2,
      "minutes": 34
    },
    "reviews": [
      {
        "name": "Jack",
        "text": "Amazing movie!"
      },
      { "name": "Tom", "text": "Super cool" }
    ]
  },
  {
    "title": "Fight Club",
    "director": "David Fincher",
    "year": 1999,
    "genres": ["drama"],
    "rating": 8.8,
    "duration": {
      "hours": 2,
      "minutes": 19
    },
    "reviews": [
      { "name": "John", "text": "10 points" },
      { "name": "Jane", "text": "Very nice" }
    ]
  },
  {
    "title": "Forrest Gump",
    "director": "Robert Zemeckis",
    "year": 1994,
    "genres": ["drama", "romance"],
    "rating": 8.8,
    "duration": {
      "hours": 2,
      "minutes": 22
    },
    "reviews": [
      { "name": "Jack", "text": "I like it!" },
      { "name": "Kate", "text": "Where does it come from?" }
    ]
  },
  {
    "title": "The Shawshank Redemption",
    "director": "Frank Darabont",
    "year": 1994,
    "genres": ["drama"],
    "rating": 9.3,
    "duration": {
      "hours": 2,
      "minutes": 22
    },
    "reviews": [
      { "name": "Kate", "text": "Where does it come from?" }
    ]
  },
  {
    "title": "The Godfather",
    "director": "Francis Ford Coppola",
    "year": 1972,
    "genres": ["crime", "drama"],
    "rating": 9.2,
    "duration": {
      "hours": 2,
      "minutes": 55
    },
    "reviews": []
  },
  {
    "title": "Goodfellas",
    "director": "Martin Scorsese",
    "year": 1990,
    "genres": ["crime", "drama", "biography"],
    "rating": 8.7,
    "duration": {
      "hours": 2,
      "minutes": 25
    },
    "reviews": []
  },
  {
    "title": "Saving Private Ryan",
    "director": "Steven Spielberg",
    "year": 1998,
    "genres": ["war", "drama"],
    "rating": 8.6,
    "duration": {
      "hours": 2,
      "minutes": 49
    },
    "reviews": [
      {
        "name": "Jerry",
        "text": "a type specimen book."
      },
      {
        "name": "Kate",
        "text": "It has survived for centuries."
      }
    ]
  },
  {
    "title": "The Hateful Eight",
    "director": "Quentin Tarantino",
    "year": 2015,
    "genres": ["crime", "drama", "mystery"],
    "rating": 7.8,
    "duration": {
      "hours": 2,
      "minutes": 48
    },
    "reviews": [
      { "name": "Din", "text": "Very boring" },
      {
        "name": "Sam",
        "text": "I like this film"
      }
    ]
  },
  {
    "title": "Inglourious Basterds",
    "director": "Quentin Tarantino",
    "year": 2009,
    "genres": ["adventure", "drama", "war"],
    "rating": 8.3,
    "duration": {
      "hours": 2,
      "minutes": 33
    },
    "reviews": [
      {
        "name": "Sam",
        "text": "I like this film"
      }
    ]
  },
  {
    "title": "Lock, Stock and Two Smoking Barrels",
    "director": "Guy Ritchie",
    "year": 1998,
    "genres": ["action", "comedy", "crime"],
    "rating": 8.2,
    "duration": {
      "hours": 1,
      "minutes": 47
    },
    "reviews": [
      { "name": "Mike", "text": "Very funny" }
    ]
  },
  {
    "title": "Snatch",
    "director": "Guy Ritchie",
    "year": 2000,
    "genres": ["comedy", "crime"],
    "rating": 8.3,
    "duration": {
      "hours": 1,
      "minutes": 44
    },
    "reviews": [
      { "name": "Jack", "text": "Cool!" }
    ]
  }
]
```

# Робота з MongoDB

**mongod** - запустити базу даних (mongod --config /usr/local/etc/mongod.conf для Mac OS)

**mongosh** - запустити MongoDB shell з терміналу

**help** - переглянути список доступних команд

**cls** - очистити MongoDB shell

**db.version()** - переглянути версію MongoDB

**db.stats()** - повертає статистику, яка відображає стан використання окремої бази даних

# Робота з БД

**show dbs** - переглянути список доступних баз

**show users** - переглянути список доступних користувачів

**show collections** - переглянути список доступних колекцій (у вигляді переліку)

**use moviebox** - перейти до бази даних "moviebox"

**db.hostInfo()** - переглянути інформацію про сервер

**db.dropDatabase()** - видалити базу даних

# Робота з колекцією

**db.getCollectionNames()** - переглянути список доступних колекцій (у вигляді масиву)

**db.movies.countDocuments()** - переглянути кількість документів у колекції

**db.getCollectionInfos()** - переглянути всю інформацію про колекцію

**db.createCollection("movies")** - створити колекцію "movies"

**db.movies.renameCollection("movies2")** - перейменувати колекцію "movies" на "movies2"

**db.movies.validate()** - перевірити колекцію на правильність

**db.movies.totalSize()** - переглянути вагу колекції у файловій системі (включаючи розмір індексів)

**db.movies.storageSize()** - переглянути вагу колекції у файловій системі (НЕ включаючи розмір індексів)

**db.movies.stats()** - переглянути повну статистику колекції

**db.movies.drop()** - видалити колекцію "movies"

**db.movies.distinct("year")** - переглянути всі унікальні дані поля "year"

# Пошук

**db.movies.find()** - знайти всі фільми колекції

**db.movies.find().pretty()** - знайти всі фільми колекції та вивести в зручному для читання форматі

**db.movies.find({ rating: null })** - знайти всі фільми, в яких відсутнє поле "rating"

**db.movies.find({ director: "Quentin Tarantino" })** - знайти фільм за полем "director"

**db.movies.find({ director: "Quentin Tarantino", rating: 8.9 })** - знайти фільм за полями "director" та "rating"

**db.movies.findOne({ \_id: ObjectId("1") })** - знайти фільми за переданим "id"

**db.movies.find({ rating: { \$gt: 9 } })** - знайти фільми з рейтингом більше 9 (не включаючи 9)

**db.movies.find({ rating: { \$gte: 9 } })** - знайти фільми з рейтингом більше 9 (включаючи 9)

**db.movies.find({ rating: { \$lt: 8 } })** - знайти фільми з рейтингом менше 8 (не включаючи 8)

**db.movies.find({ rating: { \$lte: 8 } })** - знайти фільми з рейтингом менше 8 (включаючи 8)

# Пошук

**db.movies.find({ \$or: [{ director: "Quentin Tarantino" }, { director: "Guy Ritchie" }] })** - знайти фільми режисера "Quentin Tarantino" АБО "Guy Ritchie"

**db.movies.find({ \$or: [{ director: "Quentin Tarantino" }, { rating: 9.2 }] })** - знайти фільми режисера "Quentin Tarantino" АБО фільми з рейтингом (поле "rating") 9.2

**db.movies.find({ \$and: [{ director: 'Quentin Tarantino' }, { rating: 8.3 }] })** - знайти фільми режисера "Quentin Tarantino" з рейтингом (поле "rating") 8.3

**db.movies.find({ rating: { \$in: [7.8,8.2,8.3] } })** - знайти фільми з рейтингом 7.8 АБО 8.2 АБО 8.3

**db.movies.find({ rating: { \$nin: [7.8,8.2,8.3] } })** - знайти фільми, рейтинг яких НЕ РІВНИЙ 7.8 АБО 8.2 АБО 8.3

**db.movies.find({ genres: ["drama"] })** - знайти фільми з жанром "drama"

**db.movies.find({ genres: [ "crime", "drama" ] })** - знайти фільми з жанрами "crime" і "drama" (СТРОГИЙ ПОРЯДОК)

**db.movies.find({ genres: { \$all: ["drama", "crime"] } })** - знайти фільми з жанрами "crime" і "drama" (НЕ СТРОГИЙ ПОРЯДОК)

**db.movies.find({ "reviews.name": "Jack" })** - знайти фільм з полем "reviews", в якому ІМ'Я рецензента (поле "name") - "Jack"

# Додавання

**db.movies.insertOne({ name: "Yauhen" })** - додати один документ до колекції "movies"

**db.names.insertMany([{name: "Yauhen"}, {name: "Max"}])** - додати кілька документів до колекції "names"

**db.movies.replaceOne({ id: ObjectId("629249ce4dc394dae55c7489") }, { title: "Alien" })** - повністю замінити знайдений за "id" фільм на новий з полем "title" рівним "Alien"

# Сортування

**db.movies.find().sort({ rating: -1 })** - сортувати фільми за спаданням рейтингу

**db.movies.find().sort({ title: 1 })** - сортувати фільми за заголовком в алфавітному порядку

# Обмеження

**db.movies.find({ director: "Quentin Tarantino" }).skip(1)** - знайти фільм за полем "director" і пропустити перший знайдений фільм

**db.movies.find().limit(5)** - знайти всі фільми, але повернути ТІЛЬКИ 5 перших фільмів колекції

# Видалення

**db.movies.deleteOne({ \_id: ObjectId("1") })** - видалити фільм за переданим "id"

**db.movies.deleteMany({ director: "Guy Ritchie" })** - видалити всі фільми режисера "Guy Ritchie"

# Оновлення

**db.movies.updateOne({ \_id: ObjectId('1') }, { \$set: { rating: 10, year: 1995 } })** - оновити фільм з переданим id, оновлюються поля "rating" і "year"

**db.movies.updateMany({ director: "Quentin Tarantino" }, { \$set: { rating: 10 } })** - оновити фільм режисера "Quentin Tarantino", оновлюється поле "rating"

**db.movies.updateOne({ \_id: ObjectId('1') }, { \$inc: { year: 2 } })** - оновити фільм з переданим id, оновлюється поле "year" - значення буде збільшено на 2

**db.movies.updateOne({ \_id: ObjectId('1') }, { \$inc: { year: -2 } })** - оновити фільм з переданим id, оновлюється поле "year" - значення буде зменшено на 2

**db.movies.updateOne({ \_id: ObjectId('1') }, { \$pull: { genres: "drama" } })** - оновити фільм з переданим id, з масиву "genres" буде видалено елемент "drama"

**db.movies.updateOne({ \_id: ObjectId('1') }, { \$push: { genres: "drama" } })** - оновити фільм з переданим id, до масиву "genres" буде додано елемент "drama"

**db.movies.updateOne({ \_id: ObjectId('1') }, { \$push: { genres: { \$each: ["test1", "test2"] } } })** - оновити фільм з переданим id, до масиву "genres" будуть додані елементи "test1", "test2"

# Повернення проєкції

**db.movies.find({ director: "Quentin Tarantino" }, { title: 1, director: 1 })** - знайти фільми за полем "director" і повернути дані ТІЛЬКИ з полями "title" і "director"

**db.movies.find({}, { title: 1, director: 1 })** - знайти всі фільми і повернути ТІЛЬКИ з полями "title" і "director"

**db.movies.find({}, { genres: { \$slice : 1 } })** - знайти всі фільми і повернути тільки одне ПЕРШЕ значення в масиві жанрів (поле "genres")

**db.movies.find({}, { genres: { \$slice : -1 } })** - знайти всі фільми і повернути тільки одне ОСТАННЄ значення в масиві жанрів (поле "genres")



**ДЯКУЮ ЗА УВАГУ!**